

DE4W Capstone Project

Building a Live Activity Recognition System

Orhan Konak

Table of contents

1	DE4W Capstone Project	6
2	Welcome	7
2.1	Building a Live Activity Recognition System	7
2.2	What Will We Build?	7
2.3	Project Pipeline	7
2.4	Learning Outcomes	8
2.5	Repository Structure	8
2.6	The Journey	8
3	Introduction and Motivation	10
3.1	Today We Build a System	10
3.2	Why This Matters	12
3.3	The Scenario	12
3.4	Target Activities	13
3.5	Semester Connection	13
3.6	Required Tools	13
3.7	Networking Requirement	14
3.8	Project Structure	14
3.9	How to Work Through This Project	15
3.10	Chapter Summary	15
4	Understanding the Dataset	16
4.1	Learning Objectives	16
4.2	Why Do We Start with the Data?	16
4.3	The WISDM Dataset	17
4.4	Raw File Structure	17
4.5	One Measurement	17
4.6	Understanding the Columns	17
4.7	Activity Mapping	18
4.8	Checkpoint	18
4.9	Summary	18
5	Loading and Exploring the Data	19
5.1	Learning Objectives	19
5.2	Why This Chapter Matters	19

5.3	Step 1 — Import Required Libraries	20
5.3.1	Goal	20
5.3.2	Expected Output	20
5.4	Step 2 — Configure Project Paths	20
5.4.1	Goal	20
5.4.2	Expected Output	20
5.5	Step 3 — Define Activity Mapping	21
5.5.1	Goal	21
5.5.2	Discussion	22
5.6	Step 4 — Inspect Available Files	22
5.6.1	Goal	22
5.6.2	Expected Output	22
5.7	Step 5 — Load One Participant File	23
5.7.1	Goal	23
5.7.2	Expected Output	24
5.8	Step 6 — Load All Participant Files	24
5.8.1	Goal	24
5.9	Step 7 — Inspect the Dataset	26
5.9.1	Goal	26
5.9.2	Discussion	26
5.10	Step 8 — Add Acceleration Magnitude	27
5.10.1	Goal	27
5.10.2	Discussion	27
5.11	Step 9 — Plot One Activity	27
5.11.1	Goal	27
5.11.2	Discussion	28
5.12	Step 10 — Compare Activities	29
5.12.1	Goal	29
5.12.2	Discussion	30
5.13	Step 11 — Estimate the Sampling Rate	30
5.13.1	Goal	30
5.13.2	Expected Output	30
5.14	Checkpoint	31
5.15	Mini Challenge	31
5.16	Summary	31
6	Windowing	32
6.1	Learning Objectives	32
6.2	Setup	32
6.3	Why Do We Need Windowing?	33
6.4	Step 1 — Create Windows for Walking	33
6.5	Step 2 — Inspect One Window	34
6.6	Step 3 — Visualize One Window	34

6.7	Step 4 — Create Windows for the Entire Dataset	35
6.8	Checkpoint	36
6.9	Summary	36
7	Feature Engineering	37
7.1	Learning Objectives	37
7.2	Setup	37
7.3	Why Feature Engineering?	38
7.4	Our Feature Set	39
7.5	Step 1 — Create One Example Window	39
7.6	Step 2 — Extract Features from One Window	40
	7.6.1 Expected Output	41
7.7	Step 3 — Understand the Energy Feature	41
7.8	Step 4 — Convert All Windows into Features	42
	7.8.1 Goal	42
7.9	Step 5 — Inspect the Feature Table	43
	7.9.1 Discussion	44
7.10	Step 6 — Save the Feature Table	44
7.11	Think	45
7.12	Common Pitfall	45
7.13	Checkpoint	45
7.14	Mini Challenge	46
7.15	Summary	46
8	Machine Learning	47
8.1	Learning Objectives	47
8.2	Setup	47
8.3	Why Random Forest?	48
8.4	Step 1 — Inspect the Feature Table	49
8.5	Step 2 — Define Features and Target	49
8.6	Step 3 — Random Train/Test Split	50
8.7	Step 4 — Train the Random Forest	51
8.8	Step 5 — Predict on the Test Set	67
8.9	Step 6 — Evaluate Accuracy	68
	8.9.1 Discussion	68
8.10	Step 7 — Classification Report	68
8.11	Step 8 — Confusion Matrix	69
	8.11.1 Discussion	69
8.12	Step 9 — Predict One Window	70
	8.12.1 Discussion	70
8.13	Think	70
8.14	Checkpoint	71
8.15	Mini Challenge	71

8.16 Summary	71
9 Deployment	72
10 Live Prediction	73

1 DE4W Capstone Project

2 Welcome

2.1 Building a Live Activity Recognition System

Welcome to the **DE4W Capstone Project**.

During this semester you learned many individual concepts:

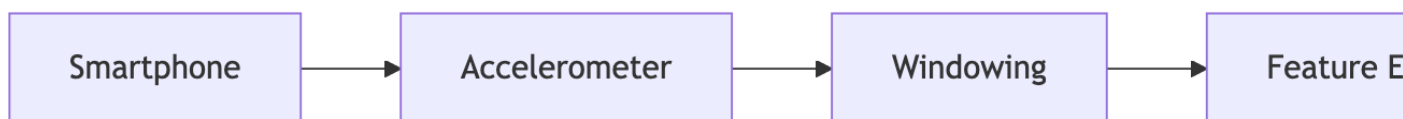
- wearable sensors
- data collection
- preprocessing
- feature engineering
- machine learning
- evaluation
- deployment

Today, we combine all of these concepts into **one complete engineering project**.

2.2 What Will We Build?

By the end of this project, you will have built a complete activity recognition system capable of recognizing human activities in real time using smartphone accelerometer data.

2.3 Project Pipeline



2.4 Learning Outcomes

After completing this project you will be able to

- build an end-to-end machine learning pipeline
 - engineer features from wearable sensor data
 - train and evaluate a machine learning model
 - deploy a trained model as a REST API
 - create an interactive dashboard
 - perform live activity recognition using your own smartphone
-

2.5 Repository Structure

```
de4w-live-activity-app/  
  
  app/  
  data/  
  docs/  
  models/  
  notebooks/  
  pyproject.toml  
  uv.lock
```

2.6 The Journey

The following chapters guide you through the complete engineering workflow.

1. Introduction
2. Dataset
3. Windowing
4. Feature Engineering
5. Machine Learning
6. Evaluation
7. Deployment

8. Live Prediction

Let's get started.

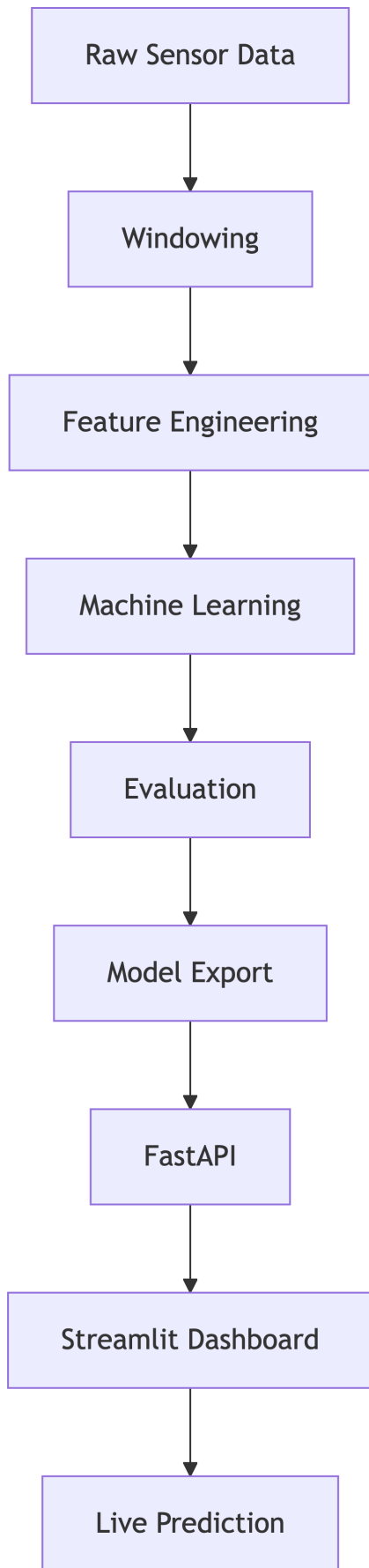
3 Introduction and Motivation

3.1 Today We Build a System

Most machine learning tutorials end after model training.

This project goes further.

We will build a complete system:



3.2 Why This Matters

A trained model is not yet an application.

A real wearable machine learning system also needs:

- reliable data handling
- reproducible preprocessing
- consistent feature extraction
- realistic evaluation
- model export
- an API
- a user interface
- live sensor integration

Engineering Insight

In real-world machine learning projects, the model is only one component of a larger system.

3.3 The Scenario

Imagine you joined a small digital health startup.

Your task:

Build a system that recognizes whether a person is walking, sitting or standing using only smartphone accelerometer data.

The system should eventually work live with a real smartphone.

3.4 Target Activities

We focus on three activities:

Activity	Why it is useful
Walking	Dynamic, periodic movement
Sitting	Low movement, common daily activity
Standing	Low movement, hard to distinguish from sitting

This simplified setup keeps the focus on the complete engineering pipeline.

3.5 Semester Connection

This project connects the entire course.

Course Topic	Where It Appears in the Project
Sensors	Smartphone accelerometer
Data Collection	WISDM and phyphox
Data Engineering	Loading, cleaning, structuring
Preprocessing	Magnitude and windowing
Feature Engineering	Statistical window features
Machine Learning	Random Forest
Evaluation	Random split vs subject-wise split
Deployment	FastAPI and Streamlit

3.6 Required Tools

You need:

- Python
- uv
- Quarto
- Jupyter kernel

- phyphox app
 - WISDM phone accelerometer data
-

3.7 Networking Requirement

For the live demo, your laptop and smartphone must be able to communicate in the same local network.

Important

University networks such as eduroam may block direct communication between devices. If phyphox is not reachable from your browser, use:

- smartphone hotspot
 - local WiFi
 - travel router
-

3.8 Project Structure

The project is organized as follows:

```
de4w-live-activity-app/  
  
  app/  
    api.py  
    config.py  
    feature_extraction.py  
    live_buffer.py  
    model_utils.py  
    phyphox_client.py  
  
  data/  
  docs/  
  models/  
  notebooks/
```

```
dashboard.py  
pyproject.toml  
uv.lock
```

3.9 How to Work Through This Project

Each major step follows the same pattern:

1. Understand the problem.
2. Implement the solution.
3. Inspect the output.
4. Discuss what could go wrong.
5. Continue to the next component.

The goal is not to copy code.

The goal is to understand the engineering decisions behind the code.

3.10 Chapter Summary

In this chapter you learned:

- what we will build
- why deployment matters
- how the project connects to the semester
- which tools are required
- why networking matters for live sensor demos
- how the project is structured

Next, we will inspect the WISDM dataset and understand the raw smartphone accelerometer signals.

4 Understanding the Dataset

4.1 Learning Objectives

After completing this chapter, you should be able to

- understand the structure of the WISDM dataset
 - explain how wearable sensor data is organized
 - identify the most important attributes of the dataset
 - understand why exploratory data analysis (EDA) is the first step of every machine learning project
-

4.2 Why Do We Start with the Data?

Many beginners believe that machine learning starts by choosing an algorithm.

In reality, every successful machine learning project starts with understanding the available data.

Before writing a single line of model training code, we should answer questions such as:

- What information is available?
- How was the data collected?
- Which sensors were used?
- What do the labels represent?
- Are there missing values?
- Are all activities equally represented?

Engineering Insight

Experienced data scientists usually spend much more time understanding and preparing data than training models.

4.3 The WISDM Dataset

Throughout this project we use the **WISDM Smartphone and Smartwatch Activity Dataset**.

For simplicity, we only use **smartphone accelerometer data**.

4.4 Raw File Structure

Each participant is stored in a separate file.

```
data_1600_accel_phone.txt
```

4.5 One Measurement

Each row represents one timestamped sensor measurement.

```
subject_id, activity_code, timestamp, x, y, z;
```

4.6 Understanding the Columns

Column	Description
subject_id	Participant identifier
activity_code	Activity label
timestamp	Measurement timestamp
x	Acceleration along x-axis
y	Acceleration along y-axis
z	Acceleration along z-axis

4.7 Activity Mapping

```
ACTIVITY_MAP = {
    'A': 'walking', 'B': 'jogging', 'C': 'stairs', 'D': 'sitting', 'E': 'standing',
    'F': 'typing', 'G': 'brushing_teeth', 'H': 'eating_soup', 'I': 'eating_chips',
    'J': 'eating_pasta', 'K': 'drinking', 'L': 'eating_sandwich', 'M': 'kicking',
    'O': 'catch', 'P': 'dribbling', 'Q': 'writing', 'R': 'clapping', 'S': 'folding_clothes'
}

TARGET_ACTIVITIES=['walking','sitting','standing']
```

i Remember

We keep the complete activity mapping although we currently use only three activities.

4.8 Checkpoint

1. What does one row represent?
 2. Why separate files per participant?
 3. Why only three activities?
 4. Why inspect the dataset before training?
-

4.9 Summary

In the next chapter we will load all participant files and inspect the raw accelerometer signals.

5 Loading and Exploring the Data

5.1 Learning Objectives

After completing this chapter, you should be able to

- load one WISDM phone accelerometer file,
 - load all participant files into one DataFrame,
 - map activity codes to readable activity names,
 - restrict the dataset to selected target activities,
 - inspect class distribution and subject distribution,
 - compute and visualize acceleration magnitude,
 - estimate the sampling rate from timestamps.
-

5.2 Why This Chapter Matters

Before we create windows, extract features, or train a model, we need to make sure that we can load and understand the raw data.

This chapter turns a folder of raw text files into a structured table that we can analyze and process further.

Engineering Insight

A clean data loading step is one of the most important parts of a reproducible machine learning project.

If the loading logic is unclear or inconsistent, every later result becomes questionable.

5.3 Step 1 — Import Required Libraries

5.3.1 Goal

We import the libraries needed for data loading, numerical processing and visualization.

```
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

5.3.2 Expected Output

This cell should run without errors.

5.4 Step 2 — Configure Project Paths

5.4.1 Goal

We define where the project root and the data directory are located.

```
PROJECT_ROOT = Path.cwd().parents[1]

DATA_DIR = PROJECT_ROOT / "data"

print("Project root:", PROJECT_ROOT)
print("Data directory:", DATA_DIR)
```

```
Project root: /Users/konak/HPI/Vorlesungen/Data Engineering for Wearables SS2026/Mini-
Project/de4w_activity_app
Data directory: /Users/konak/HPI/Vorlesungen/Data Engineering for Wearables SS2026/Mini-
Project/de4w_activity_app/data
```

5.4.2 Expected Output

The output should point to your project folder and the `data/` directory.

Common Pitfall

If the printed path does not point to your project folder, you may have started Quarto from the wrong directory.

5.5 Step 3 — Define Activity Mapping

5.5.1 Goal

The raw WISDM files use activity codes such as A, D, and E.

We map these codes to readable names.

```
ACTIVITY_MAP = {
  "A": "walking",
  "B": "jogging",
  "C": "stairs",
  "D": "sitting",
  "E": "standing",
  "F": "typing",
  "G": "brushing_teeth",
  "H": "eating_soup",
  "I": "eating_chips",
  "J": "eating_pasta",
  "K": "drinking",
  "L": "eating_sandwich",
  "M": "kicking",
  "O": "catch",
  "P": "dribbling",
  "Q": "writing",
  "R": "clapping",
  "S": "folding_clothes",
}

TARGET_ACTIVITIES = [
  "walking",
  "sitting",
  "standing",
]
```

5.5.2 Discussion

We keep the complete mapping even though we currently use only three activities.

This makes it easier to extend the project later.

5.6 Step 4 — Inspect Available Files

5.6.1 Goal

Before loading the data, we check which raw files are available.

```
files = sorted(DATA_DIR.glob("data*_accel_phone.txt"))  
  
len(files), files[:5]
```

```
(51,  
 [PosixPath('/Users/konak/HPI/Vorlesungen/Data Engineering for Wearables SS2026/Mini-  
Project/de4w_activity_app/data/data_1600_accel_phone.txt'),  
  PosixPath('/Users/konak/HPI/Vorlesungen/Data Engineering for Wearables SS2026/Mini-  
Project/de4w_activity_app/data/data_1601_accel_phone.txt'),  
  PosixPath('/Users/konak/HPI/Vorlesungen/Data Engineering for Wearables SS2026/Mini-  
Project/de4w_activity_app/data/data_1602_accel_phone.txt'),  
  PosixPath('/Users/konak/HPI/Vorlesungen/Data Engineering for Wearables SS2026/Mini-  
Project/de4w_activity_app/data/data_1603_accel_phone.txt'),  
  PosixPath('/Users/konak/HPI/Vorlesungen/Data Engineering for Wearables SS2026/Mini-  
Project/de4w_activity_app/data/data_1604_accel_phone.txt')])
```

5.6.2 Expected Output

You should see approximately 51 files, one per participant.

5.7 Step 5 — Load One Participant File

5.7.1 Goal

We first load a single file to understand the raw structure.

```
def load_single_wisdm_file(file_path: Path) -> pd.DataFrame:
    """
    Load one WISDM phone accelerometer file.

    Parameters
    -----
    file_path:
        Path to one raw WISDM phone accelerometer text file.

    Returns
    -----
    pandas.DataFrame
        DataFrame containing subject_id, activity_code, timestamp, x, y, z and activity.
    """
    df = pd.read_csv(
        file_path,
        header=None,
        names=["subject_id", "activity_code", "timestamp", "x", "y", "z"],
        sep=";",
    )

    # The last value in each row contains a trailing semicolon.
    df["z"] = (
        df["z"]
        .astype(str)
        .str.replace(";", "", regex=False)
        .astype(float)
    )

    df["activity"] = df["activity_code"].map(ACTIVITY_MAP)

    return df
```

```
df_one = load_single_wisdm_file(files[0])

df_one.head()
```

	subject_id	activity_code	timestamp	x	y	z	activity
0	1600	A	252207666810782	-0.364761	8.793503	1.055084	walking
1	1600	A	252207717164786	-0.879730	9.768784	1.016998	walking
2	1600	A	252207767518790	2.001495	11.109070	2.619156	walking
3	1600	A	252207817872794	0.450623	12.651642	0.184555	walking
4	1600	A	252207868226798	-2.164352	13.928436	-4.422485	walking

5.7.2 Expected Output

You should see a table with the columns:

```
subject_id, activity_code, timestamp, x, y, z, activity
```

5.8 Step 6 — Load All Participant Files

5.8.1 Goal

We now load all phone accelerometer files and combine them into a single DataFrame.

```
def load_wisdm_phone_accel(
    data_dir: Path,
    target_activities: list[str] | None = None,
) -> pd.DataFrame:
    """
    Load all WISDM phone accelerometer files from a directory.

    Parameters
    -----
    data_dir:
        Directory containing files named data*_accel_phone.txt.

    target_activities:
        Optional list of activity names to keep.

    Returns
    -----
    pandas.DataFrame
```



```

    Combined DataFrame containing all selected subjects and activities.
    """
    files = sorted(data_dir.glob("data*_accel_phone.txt"))

    if not files:
        raise FileNotFoundError(
            f"No WISDM phone accelerometer files found in {data_dir}."
        )

    dataframes = []

    for file_path in files:
        df_file = load_single_wisdm_file(file_path)

        if target_activities is not None:
            df_file = df_file[df_file["activity"].isin(target_activities)]

        dataframes.append(df_file)

    return pd.concat(dataframes, ignore_index=True)

```

```

df = load_wisdm_phone_accel(
    DATA_DIR,
    target_activities=TARGET_ACTIVITIES,
)

df.head()

```

	subject_id	activity_code	timestamp	x	y	z	activity
0	1600	A	252207666810782	-0.364761	8.793503	1.055084	walking
1	1600	A	252207717164786	-0.879730	9.768784	1.016998	walking
2	1600	A	252207767518790	2.001495	11.109070	2.619156	walking
3	1600	A	252207817872794	0.450623	12.651642	0.184555	walking
4	1600	A	252207868226798	-2.164352	13.928436	-4.422485	walking

5.9 Step 7 — Inspect the Dataset

5.9.1 Goal

We check the size, classes and number of subjects.

```
df.shape
```

```
(814013, 7)
```

```
df["activity"].value_counts()
```

```
activity
walking    279817
standing   269604
sitting     264592
Name: count, dtype: int64
```

```
df["subject_id"].nunique()
```

```
51
```

5.9.2 Discussion

At this point, we should verify that

- all expected subjects were loaded,
- all target activities are present,
- the class distribution is not extremely imbalanced.

5.10 Step 8 — Add Acceleration Magnitude

5.10.1 Goal

We compute the magnitude of the three-dimensional acceleration vector.

$$m = \sqrt{x^2 + y^2 + z^2}$$

```
df["magnitude"] = np.sqrt(
    df["x"] ** 2
    + df["y"] ** 2
    + df["z"] ** 2
)

df.head()
```

	subject_id	activity_code	timestamp	x	y	z	activity	magnitude
0	1600	A	252207666810782	-0.364761	8.793503	1.055084	walking	8.864082
1	1600	A	252207717164786	-0.879730	9.768784	1.016998	walking	9.860900
2	1600	A	252207767518790	2.001495	11.109070	2.619156	walking	11.587812
3	1600	A	252207817872794	0.450623	12.651642	0.184555	walking	12.661010
4	1600	A	252207868226798	-2.164352	13.928436	-4.422485	walking	14.773088

5.10.2 Discussion

The magnitude combines all three axes into one signal.

It is often easier to interpret than individual axes because it is less sensitive to phone orientation.

5.11 Step 9 — Plot One Activity

5.11.1 Goal

We visualize x, y, z and magnitude for one activity.

```

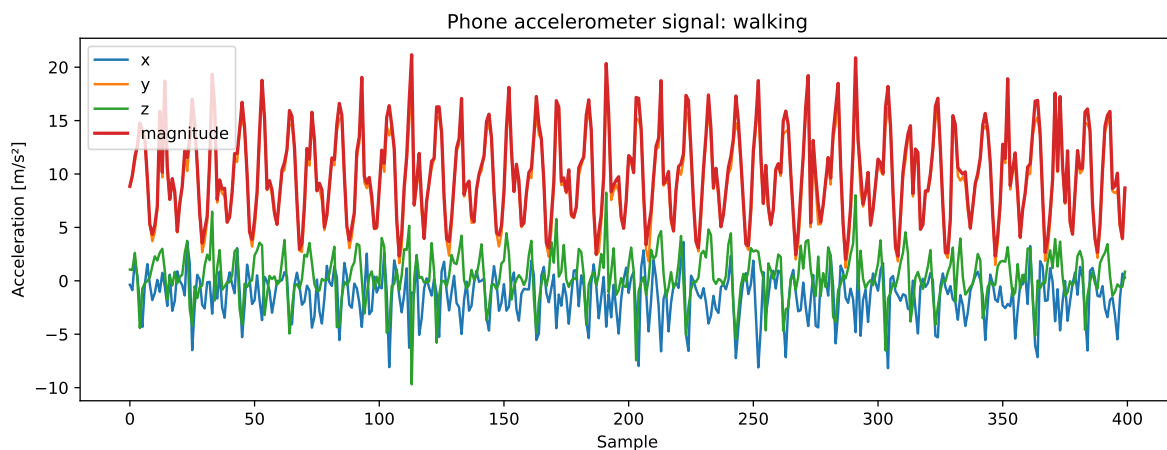
activity = "walking"

sample = (
    df[df["activity"] == activity]
    .head(400)
    .reset_index(drop=True)
)

plt.figure(figsize=(12, 4))
plt.plot(sample["x"], label="x")
plt.plot(sample["y"], label="y")
plt.plot(sample["z"], label="z")
plt.plot(sample["magnitude"], label="magnitude", linewidth=2)

plt.title(f"Phone accelerometer signal: {activity}")
plt.xlabel("Sample")
plt.ylabel("Acceleration [m/s2]")
plt.legend()
plt.show()

```



5.11.2 Discussion

Walking usually produces a more dynamic and periodic signal than sitting or standing.

5.12 Step 10 — Compare Activities

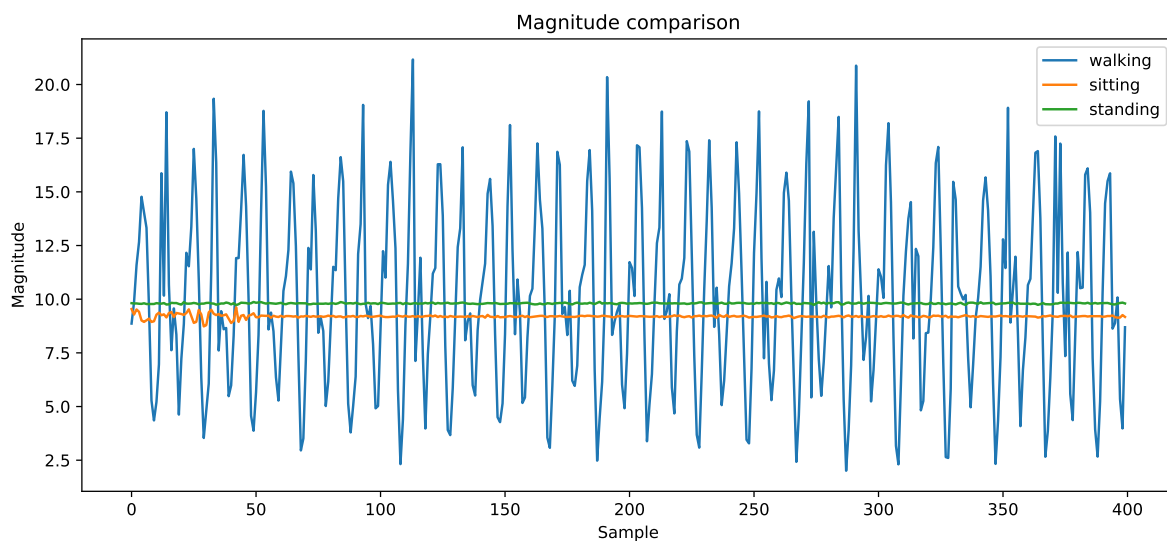
5.12.1 Goal

We compare the magnitude signal for walking, sitting and standing.

```
plt.figure(figsize=(12, 5))

for activity in TARGET_ACTIVITIES:
    signal = (
        df[df["activity"] == activity]["magnitude"]
        .head(400)
        .reset_index(drop=True)
    )
    plt.plot(signal, label=activity)

plt.title("Magnitude comparison")
plt.xlabel("Sample")
plt.ylabel("Magnitude")
plt.legend()
plt.show()
```



5.12.2 Discussion

Walking should usually be easier to distinguish because it contains stronger and more regular movement.

Sitting and standing are more similar and may be harder to separate.

5.13 Step 11 — Estimate the Sampling Rate

5.13.1 Goal

The WISDM documentation states that the data was collected at approximately 20 Hz.

We estimate the sampling rate from timestamps.

```
subject = df["subject_id"].iloc[0]
activity = "walking"

sample = df[
    (df["subject_id"] == subject)
    & (df["activity"] == activity)
].sort_values("timestamp")

dt = sample["timestamp"].diff().dropna()

sampling_rate = 1e9 / dt.mean()

sampling_rate
```

```
np.float64(19.85939378331699)
```

5.13.2 Expected Output

The result should be close to 20 Hz.

i Remember

Documentation is important, but data engineers should still verify assumptions directly from the data.

5.14 Checkpoint

Before continuing, you should be able to answer:

1. Why do we load one file before loading all files?
 2. Why do we keep the subject identifier?
 3. What is acceleration magnitude?
 4. Why are sitting and standing harder to distinguish than walking?
 5. Why is the sampling rate important for the next chapter?
-

5.15 Mini Challenge

Modify the code so that the dataset also includes `jogging`.

Which parts of the pipeline would change?

5.16 Summary

In this chapter you learned how to

- load one raw WISDM file,
- load all participant files,
- map activity codes,
- inspect class and subject distributions,
- compute acceleration magnitude,
- visualize raw signals,
- estimate sampling rate.

In the next chapter we will transform the continuous sensor stream into fixed-size windows.

6 Windowing

6.1 Learning Objectives

After completing this chapter, you should be able to

- explain why continuous sensor streams must be segmented into windows,
 - use reusable windowing code from the `app/` package,
 - create windows for one activity,
 - create windows safely per subject and activity,
 - understand how window size affects live prediction.
-

6.2 Setup

```
from pathlib import Path
import sys

import numpy as np
import matplotlib.pyplot as plt

current_path = Path.cwd().resolve()

for parent in [current_path] + list(current_path.parents):
    if (parent / "app").exists():
        PROJECT_ROOT = parent
        break
else:
    raise RuntimeError("Could not find project root containing app/ directory.")

sys.path.insert(0, str(PROJECT_ROOT))

from app.config import WINDOW_SIZE
```



```

from app.data_loader import load_wisdm_phone_accel
from app.windowing import create_windows, create_grouped_windows

DATA_DIR = PROJECT_ROOT / "data"

df = load_wisdm_phone_accel(DATA_DIR)

df["magnitude"] = np.sqrt(
    df["x"] ** 2 + df["y"] ** 2 + df["z"] ** 2
)

```

! Software Engineering Principle

This chapter does **not** redefine `create_windows()`.
 Instead, it imports reusable project code from `app.windowing`.
 This follows the **Don't Repeat Yourself (DRY)** principle.

6.3 Why Do We Need Windowing?

Wearable sensors produce a continuous stream of measurements.

Classical machine learning algorithms expect **fixed-size samples**.

Windowing bridges this gap by splitting a continuous stream into equally sized segments.

6.4 Step 1 — Create Windows for Walking

```

walking_df = (
    df[df["activity"] == "walking"]
    .sort_values("timestamp")
    .copy()
)

walking_windows = create_windows(

```

```
walking_df,
window_size=WINDOW_SIZE,
)

len(walking_windows)
```

2798

6.5 Step 2 — Inspect One Window

```
first_window = walking_windows[0]

first_window.head()
```

	subject_id	activity_code	timestamp	x	y	z	activity	magnitu
303991	1623	A	2520219142823	-0.152512	-8.585220	-1.219192	walking	8.67269
303992	1623	A	2520269496827	0.999466	-8.196548	-0.758926	walking	8.29206
303993	1623	A	2520319850830	0.450241	-8.701187	-1.290024	walking	8.80781
303994	1623	A	2520370204834	-0.042175	-9.739563	-1.787415	walking	9.90230
303995	1623	A	2520420558838	3.551483	-10.745132	-1.266403	walking	11.3874

```
first_window.shape
```

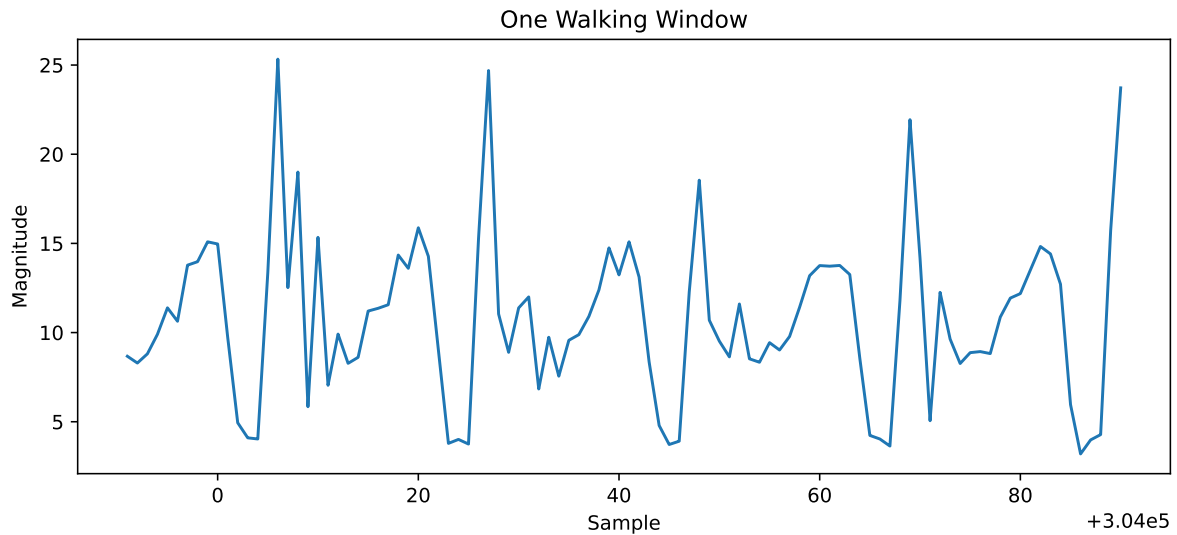
(100, 8)

6.6 Step 3 — Visualize One Window

```
plt.figure(figsize=(10, 4))

plt.plot(first_window["magnitude"])

plt.title("One Walking Window")
plt.xlabel("Sample")
plt.ylabel("Magnitude")
plt.show()
```



6.7 Step 4 — Create Windows for the Entire Dataset

```
all_windows = create_grouped_windows(
    df,
    window_size=WINDOW_SIZE,
)

len(all_windows)
```

8060

Common Pitfall

Always create windows separately for each subject and activity.
Otherwise, a window may contain multiple activities.

6.8 Checkpoint

1. Why do we need windowing?
 2. What does a 100-sample window represent at 20 Hz?
 3. Why should windows not span multiple activities?
 4. Why is the window size relevant for live prediction?
-

6.9 Summary

In this chapter you learned

- why windowing is necessary,
- how reusable project code is imported,
- how to create windows safely,
- why window size controls the trade-off between stability and responsiveness.

In the next chapter we will extract numerical features from every window.

7 Feature Engineering

7.1 Learning Objectives

After completing this chapter, you should be able to

- explain why feature engineering is needed for classical machine learning,
 - convert sensor windows into numerical feature vectors,
 - use reusable feature extraction code from the `app/` package,
 - create a complete feature table for all windows,
 - understand why training and deployment must use the same feature extraction logic.
-

7.2 Setup

```
from pathlib import Path
import sys

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

current_path = Path.cwd().resolve()

for parent in [current_path] + list(current_path.parents):
    if (parent / "app").exists():
        PROJECT_ROOT = parent
        break
else:
    raise RuntimeError("Could not find project root containing app/ directory.")

sys.path.insert(0, str(PROJECT_ROOT))
```

```

from app.config import WINDOW_SIZE
from app.data_loader import TARGET_ACTIVITIES, load_wisdm_phone_accel
from app.windowing import create_windows
from app.feature_extraction import extract_features

DATA_DIR = PROJECT_ROOT / "data"

df = load_wisdm_phone_accel(DATA_DIR)

df["magnitude"] = np.sqrt(
    df["x"] ** 2 + df["y"] ** 2 + df["z"] ** 2
)

```

! Software Engineering Principle

The feature extraction logic lives in `app/feature_extraction.py`.
The workbook imports and uses this function instead of redefining it.
This ensures that training and live prediction use the same implementation.

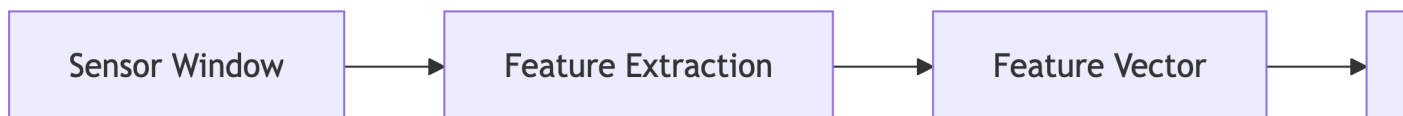
7.3 Why Feature Engineering?

A sensor window contains many raw measurements.

For example, with a window size of 100 samples and four signals (`x`, `y`, `z`, `magnitude`), one window contains 400 raw values.

Classical machine learning models such as Random Forests usually work better with structured tabular features.

Feature engineering transforms a window into a compact numerical summary.



7.4 Our Feature Set

For each signal

```
x, y, z, magnitude
```

we compute:

Feature	Meaning
mean	average signal level
std	variability
min	smallest value
max	largest value
energy	overall signal intensity

This gives

$$4 \times 5 = 20$$

features per window.

7.5 Step 1 — Create One Example Window

```
walking_df = (  
    df[df["activity"] == "walking"]  
    .sort_values("timestamp")  
    .copy()  
)  
  
windows = create_windows(  
    walking_df,  
    window_size=WINDOW_SIZE,  
)  
  
example_window = windows[0]  
  
example_window.head()
```

	subject_id	activity_code	timestamp	x	y	z	activity	magnitude
303991	1623	A	2520219142823	-0.152512	-8.585220	-1.219192	walking	8.67269
303992	1623	A	2520269496827	0.999466	-8.196548	-0.758926	walking	8.29206
303993	1623	A	2520319850830	0.450241	-8.701187	-1.290024	walking	8.80781
303994	1623	A	2520370204834	-0.042175	-9.739563	-1.787415	walking	9.90230
303995	1623	A	2520420558838	3.551483	-10.745132	-1.266403	walking	11.3874

7.6 Step 2 — Extract Features from One Window

```
example_features = extract_features(example_window)

pd.Series(example_features)
```

```
mean_x          0.719130
std_x           2.459520
min_x          -5.262344
max_x           5.583405
energy_x        650.589595
mean_y         -9.682053
std_y           3.874742
min_y         -18.635117
max_y          -2.588074
energy_y       10860.564142
mean_z         -1.843645
std_z           4.266389
min_z         -17.696304
max_z           6.950653
energy_z       2141.907743
mean_magnitude  10.765137
std_magnitude   4.566283
min_magnitude   3.196334
max_magnitude  25.331639
energy_magnitude 13653.061480
dtype: float64
```



```
len(example_features)
```

20

7.6.1 Expected Output

The output should contain 20 features.

7.7 Step 3 — Understand the Energy Feature

Energy is computed as

$$Energy = \sum_i x_i^2$$

It is often useful because dynamic activities such as walking usually produce higher signal intensity than static activities such as sitting or standing.

```
example_window["magnitude"].head()
```

```
303991    8.672698
303992    8.292063
303993    8.807811
303994    9.902309
303995   11.387479
Name: magnitude, dtype: float64
```

```
energy_magnitude = np.sum(example_window["magnitude"] ** 2)
```

```
energy_magnitude
```

```
np.float64(13653.061479592923)
```

7.8 Step 4 — Convert All Windows into Features

7.8.1 Goal

We now create one feature row per window.

Windows are created separately per subject and activity.

```
def windows_to_features(
    df: pd.DataFrame,
    window_size: int = WINDOW_SIZE,
) -> pd.DataFrame:
    """
    Convert raw sensor data into a feature table.

    Windows are created separately for each subject and activity.

    Parameters
    -----
    df:
        Sensor DataFrame containing subject_id, activity and timestamp.

    window_size:
        Number of samples per window.

    Returns
    -----
    pandas.DataFrame
        Feature table with one row per window.
    """
    rows = []

    for activity in TARGET_ACTIVITIES:
        activity_df = df[df["activity"] == activity]

        for subject_id in sorted(activity_df["subject_id"].unique()):
            subject_df = (
                activity_df[activity_df["subject_id"] == subject_id]
                .sort_values("timestamp")
            )

            windows = create_windows(
                subject_df,
```

```

        window_size=window_size,
    )

    for window in windows:
        feature_row = extract_features(window)
        feature_row["activity"] = activity
        feature_row["subject_id"] = subject_id
        rows.append(feature_row)

    return pd.DataFrame(rows)

```

```

features_df = windows_to_features(
    df,
    window_size=WINDOW_SIZE,
)

features_df.head()

```

	mean_x	std_x	min_x	max_x	energy_x	mean_y	std_y	min_y	max_y	en
0	-1.000110	1.892580	-6.492264	3.651398	454.626037	9.619472	3.986348	2.684174	18.676529	10
1	-1.344509	2.100092	-8.082993	2.628891	617.398405	9.521806	4.066769	1.647736	18.787323	10
2	-1.513662	2.453901	-8.111908	3.641586	825.258859	9.580595	4.107310	1.222855	18.672226	10
3	-1.496280	2.116861	-8.181854	3.257019	667.514539	9.551068	4.078216	1.483902	18.526184	10
4	-1.644037	2.244849	-8.546417	5.301727	769.181090	9.362707	3.838999	1.363663	18.914047	10

7.9 Step 5 — Inspect the Feature Table

```
features_df.shape
```

```
(8060, 22)
```

```
features_df["activity"].value_counts()
```

```
activity
walking    2772
standing    2670
sitting     2618
Name: count, dtype: int64
```

```
features_df["subject_id"].nunique()
```

```
51
```

```
features_df.isna().sum().sum()
```

```
np.int64(0)
```

7.9.1 Discussion

At this point, the continuous sensor data has been transformed into a tabular machine learning dataset.

Each row represents one window.

Each column represents one feature or metadata field.

7.10 Step 6 — Save the Feature Table

```
features_df.to_csv(
    DATA_DIR / "features_df.csv",
    index=False,
)
```

i Remember

Saving intermediate artifacts makes the pipeline easier to debug and reproduce.
The next chapters can load the feature table directly instead of recomputing it every time.

7.11 Think

Which feature do you expect to be most useful for distinguishing walking from standing?

- mean?
- standard deviation?
- maximum?
- energy?

Why?

7.12 Common Pitfall

Training and Deployment Must Match

If you train a model using one feature extraction method but deploy it with a different method, predictions become unreliable.

That is why the project keeps feature extraction in `app/feature_extraction.py`.

7.13 Checkpoint

Before continuing, you should be able to answer:

1. Why do we extract features from windows?
 2. Why does a Random Forest work well with feature tables?
 3. What does the energy feature measure?
 4. How many features do we compute per window?
 5. Why must training and deployment use the same feature extraction logic?
-

7.14 Mini Challenge

Add one additional feature to `app/feature_extraction.py`, for example:

- median
- range
- interquartile range

Then rerun this chapter.

How does the number of features change?

7.15 Summary

In this chapter you learned

- why feature engineering is necessary,
- how windows are transformed into feature vectors,
- how reusable project code improves maintainability,
- how to create a complete feature table,
- why consistent feature extraction is essential for deployment.

In the next chapter we will train a machine learning model using this feature table.

8 Machine Learning

8.1 Learning Objectives

After completing this chapter, you should be able to

- load the feature table created in the previous chapter,
 - define feature columns and target labels,
 - train a Random Forest classifier,
 - create predictions on unseen windows,
 - evaluate the model with accuracy and a classification report,
 - understand why a simple baseline model is valuable.
-

8.2 Setup

```
from pathlib import Path
import sys

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    ConfusionMatrixDisplay,
)
from sklearn.model_selection import train_test_split

current_path = Path.cwd().resolve()

for parent in [current_path] + list(current_path.parents):
```

```
    if (parent / "app").exists():
        PROJECT_ROOT = parent
        break
else:
    raise RuntimeError("Could not find project root containing app/ directory.")

sys.path.insert(0, str(PROJECT_ROOT))

DATA_DIR = PROJECT_ROOT / "data"

features_df = pd.read_csv(DATA_DIR / "features_df.csv")
```

8.3 Why Random Forest?

We use a Random Forest as our first model because it is a strong baseline for tabular feature data.

A Random Forest is an ensemble of decision trees.

It is useful because it is

- easy to train,
- robust,
- fast enough for this project,
- suitable for tabular data,
- relatively easy to explain.

Engineering Insight

A simple baseline model is often the best starting point.

Before using deep learning, always ask whether a simpler model already solves the problem well enough.

8.4 Step 1 — Inspect the Feature Table

```
features_df.head()
```

	mean_x	std_x	min_x	max_x	energy_x	mean_y	std_y	min_y	max_y	en
0	-1.000110	1.892580	-6.492264	3.651398	454.626037	9.619472	3.986348	2.684174	18.676529	10
1	-1.344509	2.100092	-8.082993	2.628891	617.398405	9.521806	4.066769	1.647736	18.787323	10
2	-1.513662	2.453901	-8.111908	3.641586	825.258859	9.580595	4.107310	1.222855	18.672226	10
3	-1.496280	2.116861	-8.181854	3.257019	667.514539	9.551068	4.078216	1.483902	18.526184	10
4	-1.644037	2.244849	-8.546417	5.301727	769.181090	9.362707	3.838999	1.363663	18.914047	10

```
features_df.shape
```

```
(8060, 22)
```

```
features_df["activity"].value_counts()
```

```
activity
walking      2772
standing     2670
sitting      2618
Name: count, dtype: int64
```

8.5 Step 2 — Define Features and Target

The model should learn from numerical features.

It should not use metadata such as `subject_id` as input.

```
feature_columns = [
    column
    for column in features_df.columns
    if column not in ["activity", "subject_id"]
]
```

```
X = features_df[feature_columns]
y = features_df["activity"]

X.shape, y.shape
```

```
((8060, 20), (8060,))
```

8.6 Step 3 — Random Train/Test Split

We first use a random train/test split.

This is a common first evaluation strategy.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)
```

```
X_train.shape, X_test.shape
```

```
((6448, 20), (1612, 20))
```

Common Pitfall

A random split may place windows from the same participant in both training and test sets.

This can make the evaluation look better than it really is.

We will address this in the next chapter.

8.7 Step 4 — Train the Random Forest

```
rf = RandomForestClassifier(  
    n_estimators=200,  
    random_state=42,  
)  
  
rf.fit(X_train, y_train)
```

`n_estimators` `n_estimators:` 200
int, default=100

The number of trees in the forest.

.. versionchanged:: 0.22
The default value of
“`n_estimators`” changed from
10 to 100
in 0.22.

`random_state` 42
`random_state:` int,
RandomState instance or
None, default=None

Controls both the
randomness of the
bootstrapping of the samples
used
when building trees (if
“`bootstrap=True`”) and the
sampling of the
features to consider when
looking for the best split at
each node
(if “`max_features <`
`n_features`”).
See :term:‘Glossary
<`random_state`>’ for details.

`criterion` `criterion: {"gini", "entropy", "log_loss"}, default="gini"` `'gini'`

The function to measure the quality of a split. Supported criteria are

"gini" for the Gini impurity and "log_loss" and "entropy" both for the

Shannon information gain, see :ref:'tree_mathematical_formulation'.

Note: This parameter is tree-specific.

`max_depth` `max_depth: int, None default=None`

The maximum depth of the tree. If None, then nodes are expanded until

all leaves are pure or until all leaves contain less than `min_samples_split` samples.

`min_samples_split` 2
`min_samples_split`: int or
float, default=2

The minimum number of
samples required to split an
internal node:

- If int, then consider
‘`min_samples_split`’ as the
minimum number.
- If float, then
‘`min_samples_split`’ is a
fraction and
‘`ceil(min_samples_split *
n_samples)`’ are the
minimum
number of samples for each
split.

.. versionchanged:: 0.18
Added float values for
fractions.

`min_samples_leaf`
`min_samples_leaf`: int or
float, default=1

1

The minimum number of
samples required to be at a
leaf node.

A split point at any depth
will only be considered if it
leaves at
least “`min_samples_leaf`”
training samples in each of
the left and
right branches. This may
have the effect of smoothing
the model,
especially in regression.

- If int, then consider
‘`min_samples_leaf`’ as the
minimum number.
- If float, then
‘`min_samples_leaf`’ is a
fraction and
‘`ceil(min_samples_leaf *
n_samples)`’ are the
minimum
number of samples for each
node.

.. versionchanged:: 0.18
Added float values for
fractions.

<code>min_weight_fraction_leaf</code>	<code>0.0</code>
<code>min_weight_fraction_leaf:</code>	
float, default=0.0	

The minimum weighted fraction of the sum total of weights (of all the input samples) required to be at a leaf node. Samples have equal weight when `sample_weight` is not provided.

```
max_features max_features: 'sqrt'
{"sqrt", "log2", None}, int or
float, default="sqrt"
```

The number of features to consider when looking for the best split:

- If int, then consider ‘max_features’ features at each split.
- If float, then ‘max_features’ is a fraction and ‘max(1, int(max_features * n_features_in_))’ features are considered at each split.
- If "sqrt", then ‘max_features=sqrt(n_features)’.
- If "log2", then ‘max_features=log2(n_features)’.
- If None, then ‘max_features=n_features’.

.. versionchanged:: 1.1

The default of ‘max_features’ changed from “auto” to “sqrt”.

Note: the search for a split does not stop until at least one valid partition of the node samples is found, even if it requires to effectively inspect more than “max_features” features.

<code>max_leaf_nodes</code> <code>max_leaf_nodes: int,</code> <code>default=None</code>	None
---	------

Grow trees with
“`max_leaf_nodes`” in
best-first fashion.
Best nodes are defined as
relative reduction in impurity.
If None then unlimited
number of leaf nodes.

min_impurity_decrease 0.0
min_impurity_decrease:
float, default=0.0

A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

The weighted impurity decrease equation is the following::

$$N_t / N * (\text{impurity} - N_{t_R} / N_t * \text{right_impurity} - N_{t_L} / N_t * \text{left_impurity})$$

where “N” is the total number of samples, “N_t” is the number of samples at the current node, “N_t_L” is the number of samples in the left child, and “N_t_R” is the number of samples in the right child.

“N”, “N_t”, “N_t_R” and “N_t_L” all refer to the weighted sum, if “sample_weight” is passed.

.. versionadded:: 0.19

`bootstrap` `bootstrap: bool, default=True` `True`

Whether bootstrap samples are used when building trees. If False, the whole dataset is used to build each tree.

`oob_score` `oob_score: bool or callable, default=False` `False`

Whether to use out-of-bag samples to estimate the generalization score. By default, :func:`~sklearn.metrics.accuracy\_score` is used. Provide a callable with signature `metric(y_true, y_pred)` to use a custom metric. Only available if `bootstrap=True`.

For an illustration of out-of-bag (OOB) error estimation, see the example :ref:`sphx\_glr\_auto\_examples\_ensemble\_plot\_ensemble\_oob.py`.

<code>n_jobs</code>	<code>n_jobs: int,</code> <code>default=None</code>	None
The number of jobs to run in parallel. <code>:meth:'fit'</code>, <code>:meth:'predict'</code>, <code>:meth:'decision_path'</code> and <code>:meth:'apply'</code> are all parallelized over the trees. “None” means 1 unless in a <code>:obj:'joblib.parallel_backend'</code> context. “-1” means using all processors. See <code>:term:'Glossary <n_jobs>'</code> for more details.		
<code>verbose</code>	<code>verbose: int,</code> <code>default=0</code>	0
Controls the verbosity when fitting and predicting.		
<code>warm_start</code>	<code>warm_start:</code> <code>bool, default=False</code>	False
When set to “True”, reuse the solution of the previous call to fit and add more estimators to the ensemble, otherwise, just fit a whole new forest. See <code>:term:'Glossary <warm_start>'</code> and <code>:ref:'tree_ensemble_warm_start'</code> for details.		

`class_weight` `class_weight`: None
{`"balanced"`,
`"balanced_subsample"`}, dict
or list of dicts, default=None

Weights associated with
classes in the form
“{`class_label`: `weight`}”.
If not given, all classes are
supposed to have weight one.
For
multi-output problems, a list
of dicts can be provided in
the same
order as the columns of `y`.

Note that for `multioutput`
(including `multilabel`) weights
should be
defined for each class of every
column in its own dict. For
example,
for four-class `multilabel`
classification weights should
be
[{0: 1, 1: 1}, {0: 1, 1: 5}, {0:
1, 1: 1}, {0: 1, 1: 1}] instead
of
[{1:1}, {2:5}, {3:1}, {4:1}].

The `"balanced"` mode uses
the values of `y` to
automatically adjust
weights inversely proportional
to class frequencies in the
input data
as `"n_samples / (n_classes *
np.bincount(y))"`

The `"balanced_subsample"`
mode is the same as
`"balanced"` except that
weights are computed based
on the bootstrap sample for
every tree
grown.

For multi-output, the weights
of each column of `y` will be
multiplied.

Note that these weights will
be multiplied with

<code>ccp_alpha</code>	<code>ccp_alpha:</code>	0.0
non-negative float, default=0.0		

Complexity parameter used for Minimal Cost-Complexity Pruning. The subtree with the largest cost complexity that is smaller than “`ccp_alpha`” will be chosen. By default, no pruning is performed. See :ref:‘minimal_cost_complexity_pruning’ for details. See :ref:‘sphx_glr_auto_examples_tree_plot_cost_complexity_pruning.py’ for an example of such pruning.

.. versionadded:: 0.22

`max_samples` `max_samples`: None
int or float, default=None

If bootstrap is True, the
number of samples to draw
from X
to train each base estimator.

- If None (default), then draw
'X.shape[0]' samples
irrespective of
'sample_weight'.
- If int, then draw
'max_samples' samples.
- If float, then draw
'max_samples * X.shape[0]'
unweighted samples
or 'max_samples *
sample_weight.sum()'
weighted samples.

.. versionadded:: 0.22

.. versionchanged:: 1.9
Float 'max_samples' is
relative to
'sample_weight.sum()'
instead of
'X.shape[0]' for weighted
samples.

	<code>monotonic_cst</code> <code>monotonic_cst</code> : array-like of int of shape (n_features), default=None Indicates the monotonicity constraint to enforce on each feature. - 1: monotonic increase - 0: no constraint - -1: monotonic decrease If <code>monotonic_cst</code> is None, no constraints are applied. Monotonicity constraints are not supported for: - multiclass classifications (i.e. when 'n_classes > 2'), - multioutput classifications (i.e. when 'n_outputs_ > 1'). The constraints hold over the probability of the positive class. Read more in the :ref:'User Guide <monotonic_cst_gbdtd>'. .. versionadded:: 1.4	None
--	---	------

Name	Type	Value
------	------	-------

classes__ classes__: ndarray of shape (n_classes,) or a list of such arrays	ndarray[object](3,)	['sitting','standing','walking']
---	---------------------	----------------------------------

The classes labels (single output problem), or a list of arrays of class labels (multi-output problem).

estimator__ estimator__: :class:~sklearn.tree.DecisionTreeClassifier	DecisionTreeClassifier	DecisionTreeClassifier()
--	------------------------	--------------------------

The child estimator template used to create the collection of fitted sub-estimators.

.. versionadded:: 1.2
 'base_estimator__' was renamed to 'estimator__'.
 estimators__ estimators__: list of DecisionTreeClassifier

The collection of fitted sub-estimators.

estimators__samples__ estimators__samples__: list of arrays	list
---	------

The subset of drawn samples (i.e., the in-bag samples) for each base estimator. Each subset is defined by an array of the indices selected.

.. versionadded:: 1.4

[Decision-TreeC...te=1608637542), Decision-TreeC...te=1273642419), Decision-TreeC...te=1935803228), Decision-TreeC...ate=787846414), ...]
 [array([2523, ..., dtype=int32), array([4782, ..., dtype=int32), array([5462, ..., dtype=int32), array([2848, ..., dtype=int32), ...]

<code>feature_importances_</code>	<code>ndarray[float64](20,)</code>	<code>[0.02,0.07,0.01,...,0.06,0.08,0.04]</code>
-----------------------------------	------------------------------------	--

`feature_importances_:`
 ndarray of shape
 (n_features,)

The impurity-based feature importances.
 The higher, the more important the feature.
 The importance of a feature is computed as the (normalized) total reduction of the criterion brought by that feature. It is also known as the Gini importance.

Warning: impurity-based feature importances can be misleading for high cardinality features (many unique values). See :func:‘sklearn.inspection.permutation_importance’ as an alternative.

<code>feature_names_in_</code>	<code>ndarray[object](20,)</code>	<code>['mean_x','std_x','min_x',..., 'min_magnitude', 'energy_magnitude']</code>
--------------------------------	-----------------------------------	--

`feature_names_in_:` ndarray of shape (‘n_features_in_’,)

Names of features seen during :term:‘fit’. Defined only when ‘X’ has feature names that are all strings.

.. versionadded:: 1.0

<code>n_classes__</code>	<code>n_classes_:</code>	<code>int</code>	<code>int</code>	3
or list				
The number of classes (single output problem), or a list containing the number of classes for each output (multi-output problem).				
<code>n_features_in__</code>		<code>int</code>	<code>int</code>	20
<code>n_features_in_:</code> int				
Number of features seen during :term:‘fit‘.				
.. versionadded:: 0.24				
<code>n_outputs__</code>	<code>n_outputs_:</code>	<code>int</code>	<code>int</code>	1
The number of outputs when “fit” is performed.				

8.8 Step 5 — Predict on the Test Set

```
y_pred = rf.predict(X_test)
y_pred[:10]
```

```
array(['standing', 'sitting', 'walking', 'walking', 'walking', 'walking',
       'standing', 'walking', 'walking', 'standing'], dtype=object)
```

8.9 Step 6 — Evaluate Accuracy

```
random_split_accuracy = accuracy_score(y_test, y_pred)
```

```
random_split_accuracy
```

0.989454094292804

8.9.1 Discussion

Accuracy tells us the fraction of correctly classified windows.

However, accuracy alone does not tell us which classes are confused.

8.10 Step 7 — Classification Report

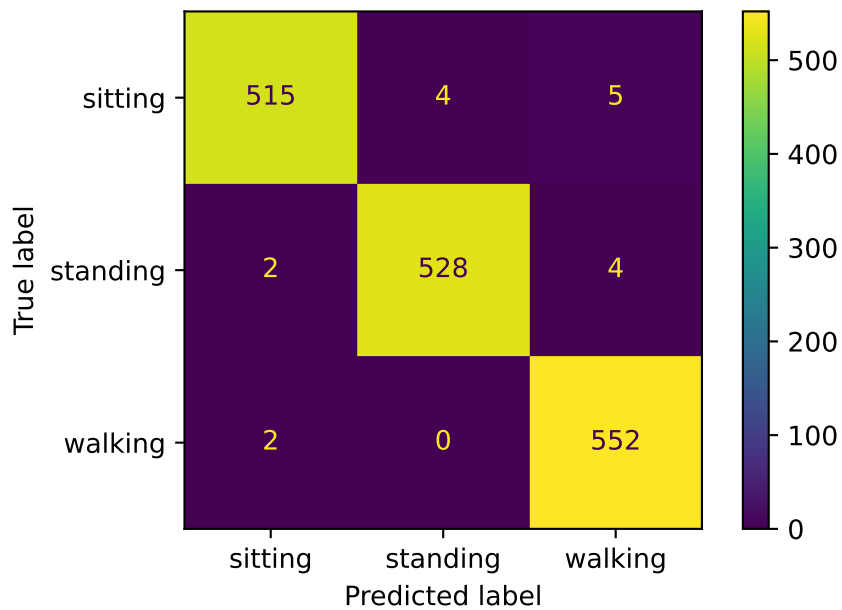
```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
sitting	0.99	0.98	0.99	524
standing	0.99	0.99	0.99	534
walking	0.98	1.00	0.99	554
accuracy			0.99	1612
macro avg	0.99	0.99	0.99	1612
weighted avg	0.99	0.99	0.99	1612

The classification report shows precision, recall and F1-score for each class.

8.11 Step 8 — Confusion Matrix

```
ConfusionMatrixDisplay.from_estimator(  
    rf,  
    X_test,  
    y_test,  
)  
  
plt.show()
```



8.11.1 Discussion

The confusion matrix helps us see which activities are confused with each other. For this dataset, walking is usually easier to classify than sitting and standing.

8.12 Step 9 — Predict One Window

```
sample = X_test.iloc[[0]]  
  
prediction = rf.predict(sample)  
  
prediction
```

```
array(['standing'], dtype=object)
```

```
pd.DataFrame(  
    rf.predict_proba(sample),  
    columns=rf.classes_,  
)
```

	sitting	standing	walking
0	0.025	0.975	0.0

8.12.1 Discussion

The model not only predicts a class but can also estimate class probabilities.

These probabilities will later be useful in the dashboard.

8.13 Think

The model may achieve very high accuracy with a random split.

Does that mean the system will work equally well on a completely new person?

Why or why not?

8.14 Checkpoint

Before continuing, you should be able to answer:

1. Why is Random Forest a good baseline model?
 2. What are X and y ?
 3. Why should `subject_id` not be used as a feature?
 4. What does accuracy measure?
 5. What additional information does the confusion matrix provide?
-

8.15 Mini Challenge

Change the number of trees from 200 to 50.

Does the accuracy change?

What does this tell you about the stability of the model?

8.16 Summary

In this chapter you learned how to

- load the feature table,
- define feature columns and target labels,
- train a Random Forest classifier,
- evaluate it with a random split,
- inspect predictions and probabilities.

In the next chapter we will ask a more realistic question:

Can the model generalize to people it has never seen before?

9 Deployment

This chapter turns the trained model into a reusable prediction service.

10 Live Prediction

This chapter connects phyphox, Streamlit, and the trained model for live activity recognition.